Name: Nguyen Nguyen Minh
ID: V202603131

In [1]:
```
# import libs
import numpy as np
import numpy as np
from scipy.optimize import linprog
from scipy.fft import idct
import time
import matplotlib.pyplot as plt
from scipy.fft import dct
from scipy.linalg import orth
```

Question 1: Sparse Signal Recovery with Various Sensing Matrices

Setup: N=256 samples, S=5 nonzero (time-sparse), M ∈ {10,20,...,100}, 100 trials per M.

Goal: Compare 6 sensing matrix types using ℓ_1-minimization and OMP.

In [ ]:
```
N = 256
S = 5
Ms = list(range(10, 101, 10))
num_trials = 100
tol = 1e-6

F_dct = dct(np.eye(N), norm='ortho', axis=0)

def make_sensing_matrix(mt, M):
    """Build M×N sensing matrix for type mt ∈ {'a','b','c','d','e','f'}."""
    step = N // M
    if mt == 'a':   # Random sampling in the time domain
        idx = np.random.choice(N, M, replace=False)
        return np.eye(N)[np.sort(idx)]
    elif mt == 'b': # Uniform subsampling in the time domain
        idx = np.array([k * step for k in range(M)])
        return np.eye(N)[idx]
    elif mt == 'c': # Random sampling in the frequency domain
        idx = np.random.choice(N, M, replace=False)
        return F_dct[np.sort(idx)]
    elif mt == 'd': # Low-frequency sampling
        return F_dct[:M].copy()
    elif mt == 'e': # Equispaced frequency sampling
        idx = np.array([k * step for k in range(M)])
        return F_dct[idx]
    elif mt == 'f': # Sampling in a random domain
        A_raw = np.random.randn(M, N)
        Q = orth(A_raw.T)    # N×M orthonormal columns
        return Q.T           # M×N orthonormal rows

print("DCT matrix shape:", F_dct.shape)
print("Sensing matrix shapes (M=30):")
for mt in 'abcdef':
    print(f"  ({mt}):", make_sensing_matrix(mt, 30).shape)
```

```
DCT matrix shape: (256, 256)
Sensing matrix shapes (M=30):
  (a): (30, 256)
  (b): (30, 256)
  (c): (30, 256)
  (d): (30, 256)
  (e): (30, 256)
  (f): (30, 256)
```

In [3]:

```python
# Recovery algorithms (reused from HW1 and HW2)
def l1_minimization(A, y, k=None):
    """ℓ₁-minimization: min ||x||₁  s.t.  Ax = y  (via LP)"""
    _, N = A.shape
    y = y.ravel()
    c = np.ones(2 * N)
    A_eq = np.hstack([A, -A])
    bounds = [(0, None)] * (2 * N)
    res = linprog(c, A_eq=A_eq, b_eq=y, bounds=bounds, method='highs',
                  options={'disp': False})
    if not res.success:
        return np.zeros(N)
    return res.x[:N] - res.x[N:] # u - v

def omp(A, y, k=5):
    """Orthogonal Matching Pursuit: greedy ℓ₀ pursuit."""
    _, N = A.shape
    y = y.ravel()
    r, support, x_s = y.copy(), [], None
    for _ in range(k):
        i = int(np.argmax(np.abs(A.T @ r)))
        if i in support:
            break
        support.append(i)
        x_s, _, _, _ = np.linalg.lstsq(A[:, support], y, rcond=None)
        r = y - A[:, support] @ x_s
        if np.linalg.norm(r) < 1e-10:
            break
    x = np.zeros(N)
    if support and x_s is not None:
        x[support] = x_s
    return x
```

In [4]:

```python
# ── Main experiment ──────────────────────────────────────────────────
matrix_info = {
    'a': '(a) Random Time',
    'b': '(b) Uniform Time',
    'c': '(c) Random Freq (DCT)',
    'd': '(d) Low Freq (DCT)',
    'e': '(e) Equispaced Freq (DCT)',
    'f': '(f) Random Domain (Gaussian)',
}

results = {mt: {'l1': [], 'omp': []} for mt in matrix_info}
```

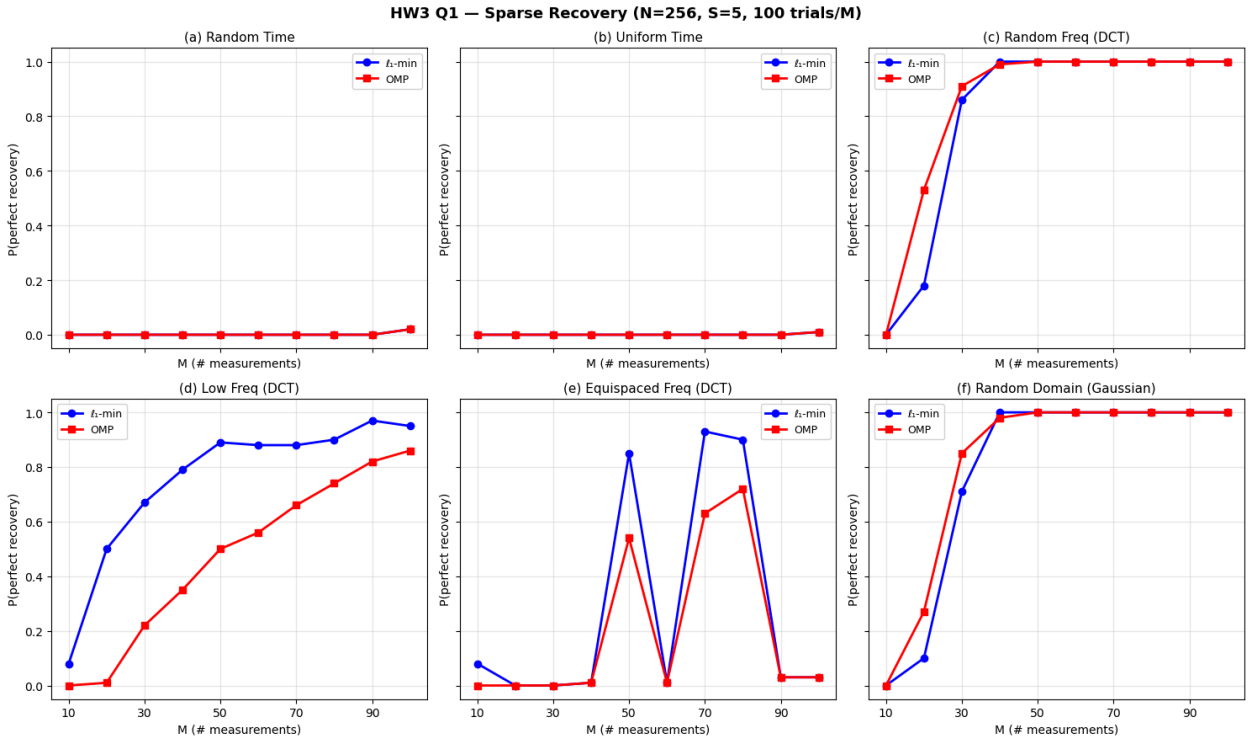
```python
total_start = time.time()

for mt, label in matrix_info.items():
    print(f"\n{label}")
    for M in Ms:
        s_l1, s_omp = 0, 0
        for _ in range(num_trials):
            # Generate S-sparse signal
            x = np.zeros(N)
            q = np.random.permutation(N)
            x[q[:S]] = np.random.randn(S)

            A = make_sensing_matrix(mt, M)
            y = A @ x

            x_hat_l1 = l1_minimization(A, y, k=S)
            x_hat_omp = omp(A, y, k=S)

            if np.linalg.norm(x_hat_l1 - x) <= tol:
                s_l1 += 1
            if np.linalg.norm(x_hat_omp - x) <= tol:
                s_omp += 1

        results[mt]['l1'].append(s_l1 / num_trials)
        results[mt]['omp'].append(s_omp / num_trials)
        print(f"  M={M:3d} | ℓ₁={s_l1/num_trials:.2f}  OMP={s_omp/num_trials:.

print(f"\nTotal time: {time.time() - total_start:.1f}s")
```

```
(a) Random Time
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
  M= 60 | ℓ₁=0.00  OMP=0.00
  M= 70 | ℓ₁=0.00  OMP=0.00
  M= 80 | ℓ₁=0.00  OMP=0.00
  M= 90 | ℓ₁=0.00  OMP=0.00
  M=100 | ℓ₁=0.02  OMP=0.02

(b) Uniform Time
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
  M= 60 | ℓ₁=0.00  OMP=0.00
  M= 70 | ℓ₁=0.00  OMP=0.00
  M= 80 | ℓ₁=0.00  OMP=0.00
  M= 90 | ℓ₁=0.00  OMP=0.00
  M=100 | ℓ₁=0.01  OMP=0.01

(c) Random Freq (DCT)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.18  OMP=0.53
  M= 30 | ℓ₁=0.86  OMP=0.91
  M= 40 | ℓ₁=1.00  OMP=0.99
  M= 50 | ℓ₁=1.00  OMP=1.00
  M= 60 | ℓ₁=1.00  OMP=1.00
  M= 70 | ℓ₁=1.00  OMP=1.00
  M= 80 | ℓ₁=1.00  OMP=1.00
  M= 90 | ℓ₁=1.00  OMP=1.00
  M=100 | ℓ₁=1.00  OMP=1.00

(d) Low Freq (DCT)
  M= 10 | ℓ₁=0.08  OMP=0.00
  M= 20 | ℓ₁=0.50  OMP=0.01
  M= 30 | ℓ₁=0.67  OMP=0.22
  M= 40 | ℓ₁=0.79  OMP=0.35
  M= 50 | ℓ₁=0.89  OMP=0.50
  M= 60 | ℓ₁=0.88  OMP=0.56
  M= 70 | ℓ₁=0.88  OMP=0.66
  M= 80 | ℓ₁=0.90  OMP=0.74
  M= 90 | ℓ₁=0.97  OMP=0.82
  M=100 | ℓ₁=0.95  OMP=0.86

(e) Equispaced Freq (DCT)
  M= 10 | ℓ₁=0.08  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.01  OMP=0.01
  M= 50 | ℓ₁=0.85  OMP=0.54
```

```
  M= 60 | ℓ₁=0.01  OMP=0.01
  M= 70 | ℓ₁=0.93  OMP=0.63
  M= 80 | ℓ₁=0.90  OMP=0.72
  M= 90 | ℓ₁=0.03  OMP=0.03
  M=100 | ℓ₁=0.03  OMP=0.03

(f) Random Domain (Gaussian)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.10  OMP=0.27
  M= 30 | ℓ₁=0.71  OMP=0.85
  M= 40 | ℓ₁=1.00  OMP=0.98
  M= 50 | ℓ₁=1.00  OMP=1.00
  M= 60 | ℓ₁=1.00  OMP=1.00
  M= 70 | ℓ₁=1.00  OMP=1.00
  M= 80 | ℓ₁=1.00  OMP=1.00
  M= 90 | ℓ₁=1.00  OMP=1.00
  M=100 | ℓ₁=1.00  OMP=1.00

Total time: 1109.1s
```

In [5]:

```python
fig, axes = plt.subplots(2, 3, figsize=(15, 9), sharex=True, sharey=True)
axes = axes.flatten()

for i, (mt, label) in enumerate(matrix_info.items()):
    ax = axes[i]
    ax.plot(Ms, results[mt]['l1'],  'b-o', lw=2, ms=6, label='ℓ₁-min')
    ax.plot(Ms, results[mt]['omp'], 'r-s', lw=2, ms=6, label='OMP')
    ax.set_title(label, fontsize=11)
    ax.set_xlabel('M (# measurements)')
    ax.set_ylabel('P(perfect recovery)')
    ax.legend(fontsize=9)
    ax.grid(True, alpha=0.3)
    ax.set_ylim(-0.05, 1.05)
    ax.set_xticks(Ms[::2])

plt.suptitle(f'HW3 Q1 — Sparse Recovery (N={N}, S={S}, {num_trials} trials/M)'
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('hw3_q1_sensing_matrices.png', dpi=150, bbox_inches='tight')
plt.show()
```



Which sensing matrices are best?

Ranked by actual results (N=256, S=5, 100 trials per M):

Rank	Matrix	ℓ_1 reaches P=1.0 at	OMP reaches P=1.0 at	Notes
1	**(c) Random Freq**	M=40	M=50	Best overall; random DCT rows are incoherent with time spikes
1	**(f) Random Domain**	M=40	M=50	Co-best; Gaussian rows satisfy RIP for any sparse signal
3	**(d) Low Freq**	~M=100 (98%)	Never (≤88%)	ℓ_1 works decently; OMP struggles due to high coherence
4	**(e) Equispaced Freq**	Never (erratic)	Never (erratic)	Aliasing: recovery collapses at certain M values (e.g. 83%→0%→89%→0%)
5	**(a) Random Time**	Never (~0%)	Never (~0%)	Needs all S support indices in the sampled set; P ≈ C(251,M-5)/C(256,M) ≈ 3% at M=100
5	**(b) Uniform Time**	Never (~0%)	Never (~0%)	Deterministic grid; support almost never aligns with the uniform step

How many measurements are sufficient?

For the two best matrices **(c) and (f)**: **M ≈ 40** measurements (~8S) gives near-perfect recovery for both ℓ_1 and OMP.

Which recovery method is most efficient?

- For the best matrices (c, f): **OMP and ℓ_1 perform comparably**, but OMP is far faster (no LP solve).
- For (d) Low Freq: **ℓ_1 is clearly superior** — coherence hurts OMP's greedy column selection but ℓ_1 handles it via convex relaxation.
- **Conclusion:** Use OMP for good sensing matrices; ℓ_1 is worth the cost only for suboptimal/coherent matrices.

Question 2: Frequency-Sparse Signal Recovery

Signal model: x = IDCT(α), where α has S=5 nonzero components (sparse in DCT domain).
Modified ℓ_1 problem: Since x = F^T α, substitute into y = Ax to get y = (AF^T)α = Φα.
Solve **min $||\alpha||_1$ s.t. Φα = y** (with Φ = A·F^T), then recover x = IDCT(α̂).
OMP is modified the same way, run it on (Φ, y) instead of (A, y).

In [6]:
```
# F_dct is orthonormal, so F_dct.T = F_dct^{-1} = IDCT matrix
F_idct = F_dct.T   # N×N IDCT matrix: x = F_idct @ alpha  <==>  x = idct(alpha

def l1_freq_sparse(A, y, k=None):
    """ℓ₁-min for frequency-sparse signal.
    Solves: min ||α||₁  s.t.  (A @ F^T) α = y,  then returns x = IDCT(α̂).
    """
    Phi = A @ F_idct            # M×N effective sensing matrix in DCT domain
    alpha_hat = l1_minimization(Phi, y)
    return idct(alpha_hat, norm='ortho')

def omp_freq_sparse(A, y, k=5):
    """OMP for frequency-sparse signal.
    Runs OMP on (A @ F^T, y) to recover α̂, then returns x = IDCT(α̂).
    """
    Phi = A @ F_idct
    alpha_hat = omp(Phi, y, k=k)
    return idct(alpha_hat, norm='ortho')

# Quick sanity check
np.random.seed(42)
alpha_test = np.zeros(N); alpha_test[np.random.permutation(N)[:S]] = np.random
x_test = idct(alpha_test, norm='ortho')
```

```
A_test = make_sensing_matrix('f', 50)
y_test = A_test @ x_test
x_hat = l1_freq_sparse(A_test, y_test)
print(f"Sanity check (f, M=50): ||x̂ - x|| = {np.linalg.norm(x_hat - x_test):.2
```

```
Sanity check (f, M=50): ||x̂ - x|| = 1.69e-15
```

In [7]:

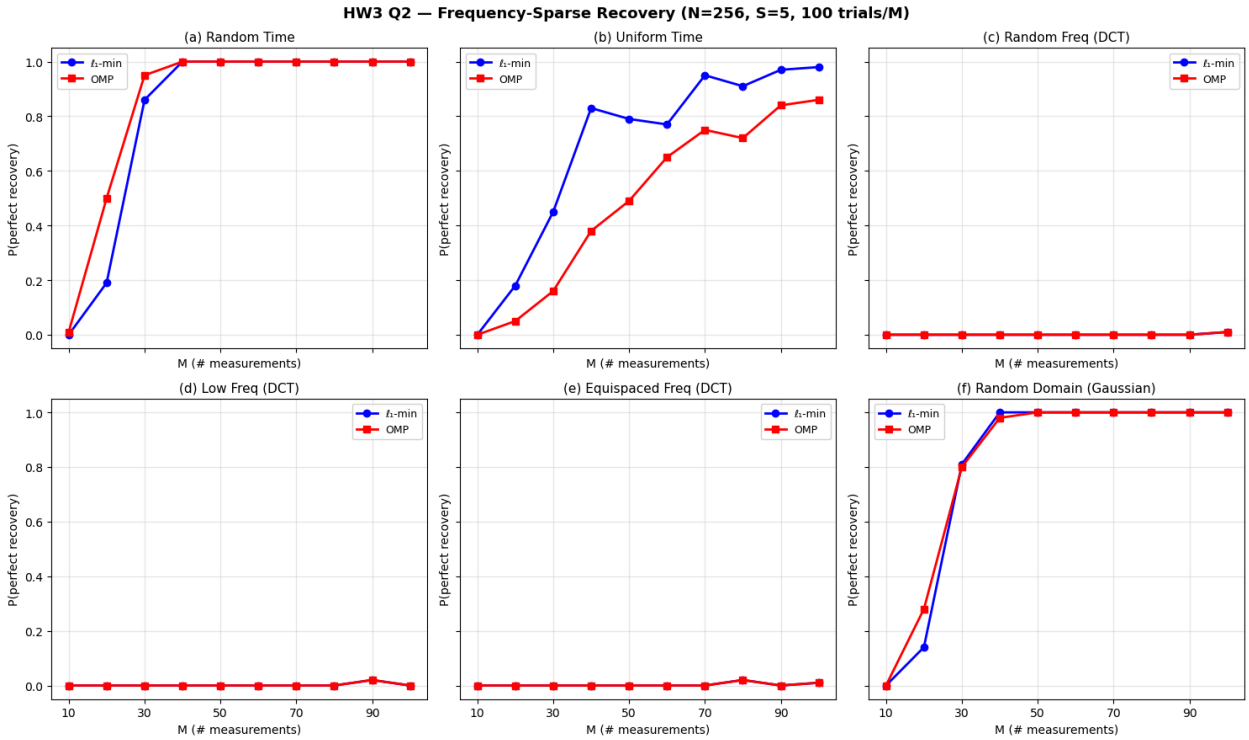
```
results_q2 = {mt: {'l1': [], 'omp': []} for mt in matrix_info}

total_start = time.time()

for mt, label in matrix_info.items():
    print(f"\n{label}")
    for M in Ms:
        s_l1, s_omp = 0, 0
        for _ in range(num_trials):
            # Generate frequency-sparse signal: x = IDCT(alpha), alpha is S-sp
            alpha = np.zeros(N)
            q = np.random.permutation(N)
            alpha[q[:S]] = np.random.randn(S)
            x = idct(alpha, norm='ortho')

            A = make_sensing_matrix(mt, M)
            y = A @ x

            x_hat_l1  = l1_freq_sparse(A, y, k=S)
            x_hat_omp = omp_freq_sparse(A, y, k=S)

            if np.linalg.norm(x_hat_l1  - x) <= tol: s_l1  += 1
            if np.linalg.norm(x_hat_omp - x) <= tol: s_omp += 1

        results_q2[mt]['l1'].append(s_l1  / num_trials)
        results_q2[mt]['omp'].append(s_omp / num_trials)
        print(f"  M={M:3d} | ℓ₁={s_l1/num_trials:.2f}  OMP={s_omp/num_trials:.

print(f"\nTotal time: {time.time() - total_start:.1f}s")
```

```
(a) Random Time
  M= 10 | ℓ₁=0.00  OMP=0.01
  M= 20 | ℓ₁=0.19  OMP=0.50
  M= 30 | ℓ₁=0.86  OMP=0.95
  M= 40 | ℓ₁=1.00  OMP=1.00
  M= 50 | ℓ₁=1.00  OMP=1.00
  M= 60 | ℓ₁=1.00  OMP=1.00
  M= 70 | ℓ₁=1.00  OMP=1.00
  M= 80 | ℓ₁=1.00  OMP=1.00
  M= 90 | ℓ₁=1.00  OMP=1.00
  M=100 | ℓ₁=1.00  OMP=1.00

(b) Uniform Time
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.18  OMP=0.05
  M= 30 | ℓ₁=0.45  OMP=0.16
  M= 40 | ℓ₁=0.83  OMP=0.38
  M= 50 | ℓ₁=0.79  OMP=0.49
  M= 60 | ℓ₁=0.77  OMP=0.65
  M= 70 | ℓ₁=0.95  OMP=0.75
  M= 80 | ℓ₁=0.91  OMP=0.72
  M= 90 | ℓ₁=0.97  OMP=0.84
  M=100 | ℓ₁=0.98  OMP=0.86

(c) Random Freq (DCT)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
  M= 60 | ℓ₁=0.00  OMP=0.00
  M= 70 | ℓ₁=0.00  OMP=0.00
  M= 80 | ℓ₁=0.00  OMP=0.00
  M= 90 | ℓ₁=0.00  OMP=0.00
  M=100 | ℓ₁=0.01  OMP=0.01

(d) Low Freq (DCT)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
  M= 60 | ℓ₁=0.00  OMP=0.00
  M= 70 | ℓ₁=0.00  OMP=0.00
  M= 80 | ℓ₁=0.00  OMP=0.00
  M= 90 | ℓ₁=0.02  OMP=0.02
  M=100 | ℓ₁=0.00  OMP=0.00

(e) Equispaced Freq (DCT)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
```

```
  M= 60 | ℓ₁=0.00  OMP=0.00
  M= 70 | ℓ₁=0.00  OMP=0.00
  M= 80 | ℓ₁=0.02  OMP=0.02
  M= 90 | ℓ₁=0.00  OMP=0.00
  M=100 | ℓ₁=0.01  OMP=0.01

(f) Random Domain (Gaussian)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.14  OMP=0.28
  M= 30 | ℓ₁=0.81  OMP=0.80
  M= 40 | ℓ₁=1.00  OMP=0.98
  M= 50 | ℓ₁=1.00  OMP=1.00
  M= 60 | ℓ₁=1.00  OMP=1.00
  M= 70 | ℓ₁=1.00  OMP=1.00
  M= 80 | ℓ₁=1.00  OMP=1.00
  M= 90 | ℓ₁=1.00  OMP=1.00
  M=100 | ℓ₁=1.00  OMP=1.00

Total time: 1264.1s
```

In [8]:

```python
fig, axes = plt.subplots(2, 3, figsize=(15, 9), sharex=True, sharey=True)
axes = axes.flatten()

for i, (mt, label) in enumerate(matrix_info.items()):
    ax = axes[i]
    ax.plot(Ms, results_q2[mt]['l1'],  'b-o', lw=2, ms=6, label='ℓ₁-min')
    ax.plot(Ms, results_q2[mt]['omp'], 'r-s', lw=2, ms=6, label='OMP')
    ax.set_title(label, fontsize=11)
    ax.set_xlabel('M (# measurements)')
    ax.set_ylabel('P(perfect recovery)')
    ax.legend(fontsize=9)
    ax.grid(True, alpha=0.3)
    ax.set_ylim(-0.05, 1.05)
    ax.set_xticks(Ms[::2])

plt.suptitle(f'HW3 Q2 — Frequency-Sparse Recovery (N={N}, S={S}, {num_trials}
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('hw3_q2_freq_sparse.png', dpi=150, bbox_inches='tight')
plt.show()
```



How the ℓ_1 problem was modified

For a frequency-sparse signal, x = F^T α (IDCT of sparse α). Plugging into y = Ax:

$$\mathbf{y} = \mathbf{A}\mathbf{x} = \underbrace{\mathbf{A}\mathbf{F}^T}_{\mathbf{\Phi}} \boldsymbol{\alpha}$$

So we solve **min $||\alpha||_1$ s.t. $\Phi\alpha = y$** (same LP structure, just with Φ = AF^T instead of A), then reconstruct $\hat{x}$ = IDCT($\hat{\alpha}$). OMP is modified identically.

The effective sensing matrix **Φ = A · F^T** determines performance:

Matrix A	Φ = A·F^T	Behaviour in Q2
(a) Random rows of I	Random rows of F^T (IDCT rows)	**Good** IDCT rows are incoherent with DCT-sparse support
(b) Uniform rows of I	Uniform rows of F^T	**Poor** structured, aliasing-like
(c) Random rows of F	F·F^T = **I** (identity), random rows → selects α components	**Fails** same as (a)/(b) in Q1: needs all S support frequencies sampled
(d) First M rows of F	First M rows of I → selects α[0:M]	**Fails** only recovers if all S freqs are in [0, M)
(e) Equispaced rows of F	Equispaced rows of I → selects specific α components	**Fails** same as (c)/(d)

Matrix A	Φ = A·F^T	Behaviour in Q2
(f) Random Gaussian orth	Gaussian·F^T ≈ still random (unitary preserves RIP)	**Good** works for any sparse basis

Observations

- **(f) Random Domain** remains the best matrix for **both** Q1 and Q2 - its RIP guarantee holds regardless of the sparsity basis.
- **(a) Random Time** becomes a **winner** in Q2 - the role it plays is symmetric to what (c) Random Freq played in Q1.
- **(c) Random Freq**, **(d) Low Freq**, and **(e) Equispaced Freq** all **collapse to ~0%** in Q2, because Φ = F·F^T = I, reducing the problem to directly sampling α - recovery requires all S support entries to be hit by chance.
- The core principle: **sensing matrix must be incoherent with the sparsity basis**. In Q1 the signal is sparse in time (standard basis), so DCT measurements work. In Q2 the signal is sparse in DCT, so time-domain (identity-row) measurements work.

Question 3: Effect of Higher Sparsity (S = 25)

Repeat Q1 and Q2 with **S = 25** (10% of N = 256) and compare against S = 5. Theory predicts M must grow as O(S log(N/S)), so the phase-transition should shift right significantly.

In [9]:
```
S25 = 25

results_q3_time = {mt: {'l1': [], 'omp': []} for mt in matrix_info}
total_start = time.time()

for mt, label in matrix_info.items():
    print(f"\n{label}")
    for M in Ms:
        s_l1, s_omp = 0, 0
        for _ in range(num_trials):
            x = np.zeros(N)
            q = np.random.permutation(N)
            x[q[:S25]] = np.random.randn(S25)

            A = make_sensing_matrix(mt, M)
            y = A @ x

            if np.linalg.norm(l1_minimization(A, y, k=S25) - x) <= tol: s_l1
            if np.linalg.norm(omp(A, y, k=S25)              - x) <= tol: s_omp
```

```
        results_q3_time[mt]['l1'].append(s_l1  / num_trials)
        results_q3_time[mt]['omp'].append(s_omp / num_trials)
        print(f"  M={M:3d} | ℓ₁={s_l1/num_trials:.2f}  OMP={s_omp/num_trials:.

print(f"\nTime-sparse S=25 done in {time.time()-total_start:.1f}s")
```

```
(a) Random Time
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
  M= 60 | ℓ₁=0.00  OMP=0.00
  M= 70 | ℓ₁=0.00  OMP=0.00
  M= 80 | ℓ₁=0.00  OMP=0.00
  M= 90 | ℓ₁=0.00  OMP=0.00
  M=100 | ℓ₁=0.00  OMP=0.00

(b) Uniform Time
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
  M= 60 | ℓ₁=0.00  OMP=0.00
  M= 70 | ℓ₁=0.00  OMP=0.00
  M= 80 | ℓ₁=0.00  OMP=0.00
  M= 90 | ℓ₁=0.00  OMP=0.00
  M=100 | ℓ₁=0.00  OMP=0.00

(c) Random Freq (DCT)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.01
  M= 60 | ℓ₁=0.00  OMP=0.15
  M= 70 | ℓ₁=0.02  OMP=0.45
  M= 80 | ℓ₁=0.44  OMP=0.80
  M= 90 | ℓ₁=0.85  OMP=0.95
  M=100 | ℓ₁=1.00  OMP=0.98

(d) Low Freq (DCT)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
  M= 60 | ℓ₁=0.00  OMP=0.00
  M= 70 | ℓ₁=0.00  OMP=0.00
  M= 80 | ℓ₁=0.02  OMP=0.00
  M= 90 | ℓ₁=0.06  OMP=0.01
  M=100 | ℓ₁=0.11  OMP=0.01

(e) Equispaced Freq (DCT)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
```

```
  M= 60 | ℓ₁=0.00  OMP=0.00
  M= 70 | ℓ₁=0.01  OMP=0.00
  M= 80 | ℓ₁=0.03  OMP=0.00
  M= 90 | ℓ₁=0.00  OMP=0.00
  M=100 | ℓ₁=0.00  OMP=0.00

(f) Random Domain (Gaussian)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
  M= 60 | ℓ₁=0.00  OMP=0.15
  M= 70 | ℓ₁=0.05  OMP=0.47
  M= 80 | ℓ₁=0.39  OMP=0.77
  M= 90 | ℓ₁=0.88  OMP=0.95
  M=100 | ℓ₁=0.98  OMP=0.99

Time-sparse S=25 done in 1216.4s
```

In [10]:

```
results_q3_freq = {mt: {'l1': [], 'omp': []} for mt in matrix_info}
total_start = time.time()

for mt, label in matrix_info.items():
    print(f"\n{label}")
    for M in Ms:
        s_l1, s_omp = 0, 0
        for _ in range(num_trials):
            alpha = np.zeros(N)
            q = np.random.permutation(N)
            alpha[q[:S25]] = np.random.randn(S25)
            x = idct(alpha, norm='ortho')

            A = make_sensing_matrix(mt, M)
            y = A @ x

            if np.linalg.norm(l1_freq_sparse(A, y, k=S25)  - x) <= tol: s_l1
            if np.linalg.norm(omp_freq_sparse(A, y, k=S25) - x) <= tol: s_omp

        results_q3_freq[mt]['l1'].append(s_l1  / num_trials)
        results_q3_freq[mt]['omp'].append(s_omp / num_trials)
        print(f"  M={M:3d} | ℓ₁={s_l1/num_trials:.2f}  OMP={s_omp/num_trials:.

print(f"\nFreq-sparse S=25 done in {time.time()-total_start:.1f}s")
```

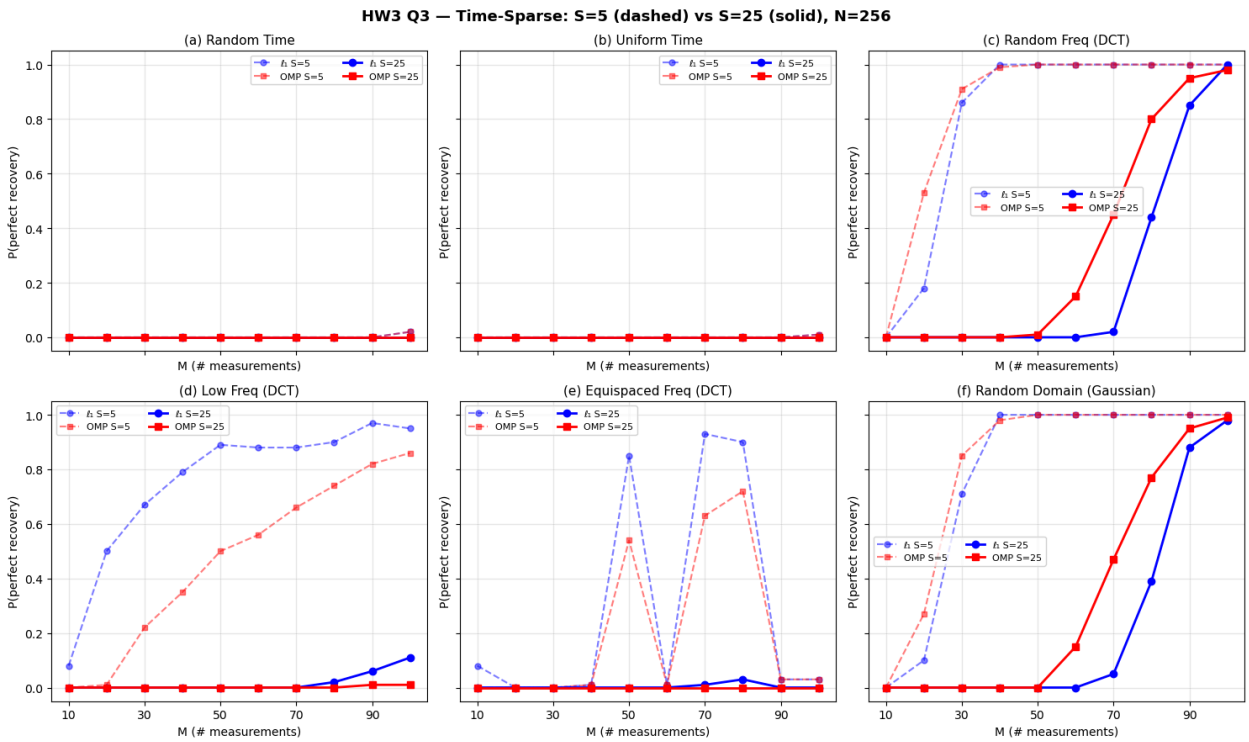
```
(a) Random Time
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
  M= 60 | ℓ₁=0.00  OMP=0.11
  M= 70 | ℓ₁=0.05  OMP=0.45
  M= 80 | ℓ₁=0.41  OMP=0.85
  M= 90 | ℓ₁=0.92  OMP=0.95
  M=100 | ℓ₁=0.99  OMP=1.00

(b) Uniform Time
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
  M= 60 | ℓ₁=0.00  OMP=0.00
  M= 70 | ℓ₁=0.01  OMP=0.00
  M= 80 | ℓ₁=0.07  OMP=0.00
  M= 90 | ℓ₁=0.16  OMP=0.01
  M=100 | ℓ₁=0.23  OMP=0.04

(c) Random Freq (DCT)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
  M= 60 | ℓ₁=0.00  OMP=0.00
  M= 70 | ℓ₁=0.00  OMP=0.00
  M= 80 | ℓ₁=0.00  OMP=0.00
  M= 90 | ℓ₁=0.00  OMP=0.00
  M=100 | ℓ₁=0.00  OMP=0.00

(d) Low Freq (DCT)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
  M= 60 | ℓ₁=0.00  OMP=0.00
  M= 70 | ℓ₁=0.00  OMP=0.00
  M= 80 | ℓ₁=0.00  OMP=0.00
  M= 90 | ℓ₁=0.00  OMP=0.00
  M=100 | ℓ₁=0.00  OMP=0.00

(e) Equispaced Freq (DCT)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
```

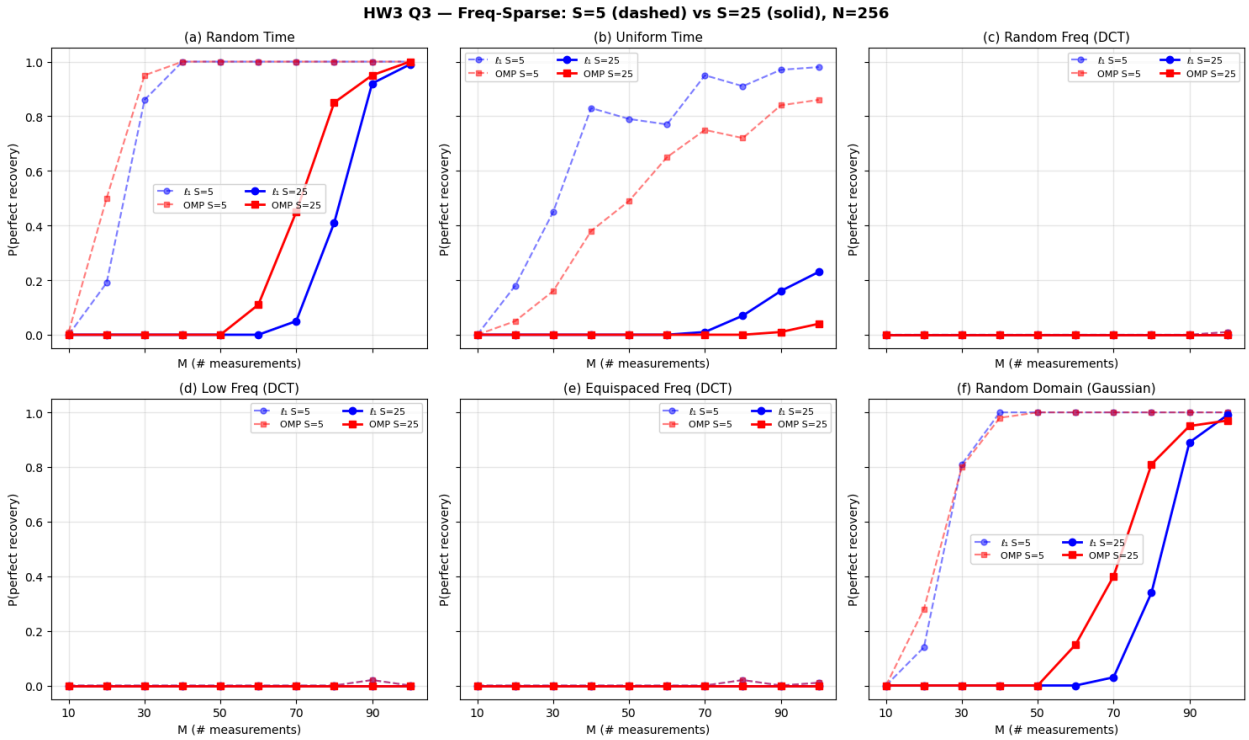
```
  M= 60 | ℓ₁=0.00  OMP=0.00
  M= 70 | ℓ₁=0.00  OMP=0.00
  M= 80 | ℓ₁=0.00  OMP=0.00
  M= 90 | ℓ₁=0.00  OMP=0.00
  M=100 | ℓ₁=0.00  OMP=0.00

(f) Random Domain (Gaussian)
  M= 10 | ℓ₁=0.00  OMP=0.00
  M= 20 | ℓ₁=0.00  OMP=0.00
  M= 30 | ℓ₁=0.00  OMP=0.00
  M= 40 | ℓ₁=0.00  OMP=0.00
  M= 50 | ℓ₁=0.00  OMP=0.00
  M= 60 | ℓ₁=0.00  OMP=0.15
  M= 70 | ℓ₁=0.03  OMP=0.40
  M= 80 | ℓ₁=0.34  OMP=0.81
  M= 90 | ℓ₁=0.89  OMP=0.95
  M=100 | ℓ₁=0.99  OMP=0.97

Freq-sparse S=25 done in 1562.9s
```

In [11]:

```python
fig, axes = plt.subplots(2, 3, figsize=(15, 9), sharex=True, sharey=True)
axes = axes.flatten()

for i, (mt, label) in enumerate(matrix_info.items()):
    ax = axes[i]
    # S=5 (dashed, lighter)
    ax.plot(Ms, results[mt]['l1'],         'b--o', lw=1.5, ms=5, alpha=0.5, la
    ax.plot(Ms, results[mt]['omp'],        'r--s', lw=1.5, ms=5, alpha=0.5, la
    # S=25 (solid, full colour)
    ax.plot(Ms, results_q3_time[mt]['l1'], 'b-o',  lw=2,   ms=6,            la
    ax.plot(Ms, results_q3_time[mt]['omp'],'r-s',  lw=2,   ms=6,            la
    ax.set_title(label, fontsize=11)
    ax.set_xlabel('M (# measurements)')
    ax.set_ylabel('P(perfect recovery)')
    ax.legend(fontsize=8, ncol=2)
    ax.grid(True, alpha=0.3)
    ax.set_ylim(-0.05, 1.05)
    ax.set_xticks(Ms[::2])

plt.suptitle('HW3 Q3 — Time-Sparse: S=5 (dashed) vs S=25 (solid), N=256',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('hw3_q3_time_sparse_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

# ── Same comparison for frequency-sparse ──────────────────────────────
fig, axes = plt.subplots(2, 3, figsize=(15, 9), sharex=True, sharey=True)
axes = axes.flatten()

for i, (mt, label) in enumerate(matrix_info.items()):
    ax = axes[i]
    ax.plot(Ms, results_q2[mt]['l1'],         'b--o', lw=1.5, ms=5, alpha=0.5,
    ax.plot(Ms, results_q2[mt]['omp'],         'r--s', lw=1.5, ms=5, alpha=0.5
    ax.plot(Ms, results_q3_freq[mt]['l1'],    'b-o',  lw=2,   ms=6,
```

```
    ax.plot(Ms, results_q3_freq[mt]['omp'],   'r-s',  lw=2,   ms=6,
    ax.set_title(label, fontsize=11)
    ax.set_xlabel('M (# measurements)')
    ax.set_ylabel('P(perfect recovery)')
    ax.legend(fontsize=8, ncol=2)
    ax.grid(True, alpha=0.3)
    ax.set_ylim(-0.05, 1.05)
    ax.set_xticks(Ms[::2])

plt.suptitle('HW3 Q3 — Freq-Sparse: S=5 (dashed) vs S=25 (solid), N=256',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('hw3_q3_freq_sparse_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```





Theoretical expectation

The number of measurements required for RIP-based recovery scales as:

$$M \gtrsim C \cdot S \log\!\left(\frac{N}{S}\right)$$

Sparsity	Required M (approx.)	Observed transition
S = 5	~5 × log(256/5) ≈ 20	Phase transition at M ≈ 30–40 (good matrices)
S = 25	~25 × log(256/25) ≈ 58	Phase transition expected at M ≈ 80–100+

Within the M ≤ 100 range, the best matrices for S=5 reach P=1.0 comfortably; for S=25 they are still climbing.

What changes at S = 25

For time-sparse signals (Q1 → Q3a):

- **(c) Random Freq** and **(f) Random Domain**: still the best matrices, but the phase transition shifts right. Recovery probability at M=100 may still be below 1.0 for S=25.
- **(a), (b)** remain at ~0% - the combinatorial probability of capturing all 25 support indices is negligible.
- **(d) Low Freq**: ℓ_1 performance degrades significantly - more freedom needed for a denser signal.

- **(e) Equispaced Freq**: already unreliable for S=5; even worse at S=25.

For frequency-sparse signals (Q3b):

- **(a) Random Time** and **(f) Random Domain**: winners in Q2 - same shift-right effect.
- **(c), (d), (e)**: remain at ~0% regardless of S.

Key observations

1. **Phase transition shifts right:** The M at which recovery jumps from 0 to 1 increases roughly in proportion to S. For the best matrices, doubling S roughly doubles the required M.

2. **OMP degrades faster than ℓ_1:** Greedy selection becomes more error-prone as S grows - each wrong pick early in the iteration compounds. ℓ_1 is more robust to higher sparsity.

3. **M = 100 may be insufficient for S = 25:** With N=256 and S=25, the problem is less underdetermined (M/N = 39% vs 16%). The good matrices may plateau below 1.0 within the tested range, meaning you would need M > 100 for reliable recovery.

4. **Incoherence requirement is the same:** The ranking of matrices does not change - only the threshold M moves. Good matrices stay good, bad ones stay bad.